

Modular LabJack GUI Framework

System Adaptation Procedure

BYU TIPICE Laboratory
Document version 1.0 | May 2026

Overview

This procedure describes how to adapt the Modular LabJack GUI Framework to a new laboratory system. The framework is designed so that hardware-specific details are isolated in two files only — **config.py** and **app.py**. All other modules (**sensor.py**, **control_loop.py**, **pid_controller.py**, **labjack_interface.py**, **ui_builders.py**, **data_logger.py**) are fully reusable and should never require modification.

For the majority of new systems, only **config.py** needs to be edited. The file **app.py** is only modified when the GUI panel layout changes or system-specific hardware must be added or removed.

Files — What to Edit and When

File	Edit?	Purpose
config.py	Always	Pin assignments, calibrations, PID defaults, log columns
app.py	Layout changes	Panel arrangement, system-specific hardware frames
main.py	Never	Entry point — launches the app
sensor.py	Never	Sensor dataclass
control_loop.py	Never	Control loop dataclass
pid_controller.py	Never	Discrete PID with anti-windup
labjack_interface.py	Never	LabJack T7 hardware abstraction
ui_builders.py	New panel types	Reusable widget-building functions
data_logger.py	Never	Background-thread CSV logger

Step 1 — Copy the Project Folder

Duplicate the entire **packed_columns/** folder and rename it for your new system. The folder structure should be preserved exactly as shown below.

```
distillation_column/
■■■ main.py
■■■ app.py
■■■ config.py
■■■ sensor.py
■■■ control_loop.py
■■■ pid_controller.py
■■■ labjack_interface.py
■■■ ui_builders.py
■■■ data_logger.py
■■■ assets/
■■■ your_logo.png
■■■ forest-dark.tcl
■■■ forest-light.tcl
```

Step 2 — Edit config.py

config.py is the single file that describes your hardware. Work through each subsection below in order.

2a — System Identity

Set the window title, logo path, and output folder name.

```
SYSTEM_NAME = "Distillation Column" # shown in the window title bar
LOGO_FILE = "assets/your_logo.png"
LOG_FOLDER = "DistillationColumn_LoggedData"
```

2b — LabJack Ethernet Address

Each LabJack T7 on the network has a unique IP address. Update this to match the address assigned to this system's unit.

```
ETHERNET_ADDRESS = "10.8.112.XX" # your T7's IP address
```

2c — Fixed Digital and Special Outputs

These constants map system relay and analog output pins. Keep, remove, or rename them to match your hardware. If a constant is removed here, also remove the code that references it in app.py (see Step 3b).

```
MAIN_POWER_PIN = "FI06" # digital pin driving the main relay
COLUMN_SELECTOR_PIN = "FI07" # remove if no selector relay
AIR_SETPOINT_PIN = "TDAC0" # remove if no air MFC
AIR_SETPOINT_SCALE = 200.0 # SLPM / scale = voltage to MFC
```

2d — Read-Only Sensor Displays (SENSOR_CONFIGS)

Each entry in this dictionary creates one read-only display panel in the GUI. Add one entry for every sensor that has no PID control loop. Setting **pin** to an empty string marks the channel as unassigned — the display will show "Error" until a pin is provided.

```

SENSOR_CONFIGS = {
  "reboiler_temperature": {
    "label": "Reboiler Temp",
    "unit": "C",
    "pin": "AIN3",
    "calibration": lambda v: 52.1 * v - 10.4,
  },
  "condenser_level": {
    "label": "Condenser Level",
    "unit": "mm",
    "pin": "", # not yet wired
    "calibration": lambda v: v,
  },
}

```

Deriving a Linear Calibration Equation

For sensors with a linear voltage output, use the datasheet minimum and maximum values to compute slope and intercept:

```

slope = (eng_max - eng_min) / (V_max - V_min)
intercept = eng_min - slope * V_min

# Example: 4-20 mA sensor (via 250 ohm shunt: 1-5 V) reading 0-100 PSI
# slope = (100 - 0) / (5.0 - 1.0) = 25.0
# intercept = 0 - 25.0 * 1.0 = -25.0
"calibration": lambda v: 25.0 * v - 25.0

```

2e — PID Control Loops (CONTROL_LOOP_CONFIGS)

Each entry creates a full control panel with setpoint spinbox, live reading, manual/auto switch, valve slider with progress bar, and Kc/Ti/Td tuning spinboxes. The optional **extra_sensor_key** embeds a sensor reading inside the panel.

```

CONTROL_LOOP_CONFIGS = {
  "reflux_flow": {
    "label": "Reflux Flow",
    "unit": "L/min",
    "input_pin": "AIN0",
    "output_pin": "DAC0",
    "calibration": lambda v: 15.2 * v - 3.1,
    "setpoint_min": 0,
    "setpoint_max": 20,
    "default_setpoint": 0,
    "pid_defaults": {"Kc": 0.1, "Ti": 0.5, "Td": 0.0},
    "extra_sensor_key": "reboiler_temperature", # optional
  },
}

```

PID Initial Tuning Guidance

Parameter	Meaning	Starting Point
Kc	Proportional gain. Use a negative value for reverse-acting loops (e.g. level control where high level should close a valve).	$\pm 0.1 - 0.5$
Ti	Integral time in minutes. Larger value = slower integral action.	0.5 – 5.0
Td	Derivative time in minutes. Leave at 0.0 unless process has significant lag.	0.0

2f — Data Logging Columns (LOG_COLUMNS)

List exactly which values should appear in the CSV output, in order. Each key must exactly match an entry in SENSOR_CONFIGS (kind = "sensor") or CONTROL_LOOP_CONFIGS (kind = "loop").

```
LOG_COLUMNS = {
    "Reboiler Temp (C)": ("sensor", "reboiler_temperature"),
    "Reflux Flow (L/min)": ("loop", "reflux_flow"),
}
```

Step 3 — Edit app.py (Only if Needed)

app.py only needs to be modified for the two reasons described below. If config.py already covers all your changes, skip this step entirely.

3a — Panel Layout Changed

The **_build_ui** method controls where each control loop panel sits on screen. Locate the **loop_rows** dictionary and update it to match your loops. Each key must match a key in CONTROL_LOOP_CONFIGS; the integer value is the tkinter grid row.

```
loop_rows = {
    "reflux_flow": 0, # row 0, right-hand side
    "bottoms_flow": 1, # row 1, right-hand side
}
```

3b — Removing Packed-Columns-Specific Hardware

If your system does not have an air MFC, column selector relay, or other packed-columns-specific hardware, remove the corresponding method and its call site.

Hardware to Remove	Method in app.py	Also Remove From
Air MFC frame	<code>_build_air_flow_frame()</code>	config.py AIR_SETPOINT_PIN / SCALE
Column selector radios	Lines in <code>_build_controls_frame()</code>	config.py COLUMN_SELECTOR_PIN
Air setpoint output	<code>_update_air_setpoint_output()</code>	<code>_schedule_updates()</code> call

3c — Adding New System-Specific Hardware

If your system has something unique (e.g. a heater on/off switch, a pump speed dial) that does not fit the standard sensor or PID panel pattern, add a new frame-builder method in `app.py` following the same pattern as `_build_air_flow_frame()`, then call it from `_build_ui()`.

Step 4 — Replace Assets

Replace the files in the **assets/** folder with your own. The theme files (`forest-dark.tcl`, `forest-light.tcl`) are not system-specific and should be copied unchanged from the packed columns folder.

File	Action
<code>your_logo.png</code>	Replace with your lab/system logo. Update <code>LOGO_FILE</code> in <code>config.py</code> to match the filename.
<code>forest-dark.tcl</code>	Copy unchanged from the packed columns assets folder.
<code>forest-light.tcl</code>	Copy unchanged from the packed columns assets folder.

Step 5 — Run and Verify

Launch the GUI from a terminal:

```
cd distillation_column
python3 main.py
```

Pre-Connection Checks (LabJack Disconnected)

- Window title bar shows your `SYSTEM_NAME`.
- All controls and spinboxes are greyed out.
- The correct number of sensor display panels appears.
- The correct number of control loop panels appears, each with the right label and unit.

Post-Connection Checks

- Connect via USB or Ethernet; the status label turns green.
- Turn on Main Power; all controls unlock.
- Sensor entries show live readings, or "Error" for any unassigned pins.
- Set a manual valve output on a control loop; confirm the progress bar moves.
- Switch a loop to AUTO mode; confirm the PID output responds to a setpoint change.
- Start Logging; confirm a timestamped CSV appears in your `LOG_FOLDER` directory.
- Open the CSV and verify column headers match your `LOG_COLUMNS` definition.

Quick Reference

Change Needed	File to Edit	What to Modify
Add a new read-only sensor	<code>config.py</code>	Add entry to <code>SENSOR_CONFIGS</code>
Add a new PID control loop	<code>config.py</code>	Add entry to <code>CONTROL_LOOP_CONFIGS</code>
Change the LabJack IP address	<code>config.py</code>	<code>ETHERNET_ADDRESS</code>
Change the main power relay pin	<code>config.py</code>	<code>MAIN_POWER_PIN</code>
Add or remove a CSV log column	<code>config.py</code>	<code>LOG_COLUMNS</code>
Rearrange panel positions	<code>app.py</code>	<code>loop_rows</code> dict in <code>_build_ui()</code>
Remove air MFC / column selector	<code>config.py</code> + <code>app.py</code>	See Step 3b
Add unique hardware frame	<code>app.py</code>	New <code>_build_*_frame()</code> method
Add a new panel type	<code>ui_builders.py</code>	New builder function
Swap LabJack model (T4, etc.)	<code>labjack_interface.py</code>	Model string in <code>connect()</code>

For questions about this framework contact the BYU TIPICE laboratory.